SyncPay

College Project Documentation + External Viva Notes

Offline-first Digital Wallet with QR Payments and Central Ledger

Prepared for team revision: architecture, frontend, backend, database schema, flows, setup steps, queries, and viva answers.

> **One-line explanation:** SyncPay ek offline-first digital wallet system hai jisme user QR scan karke payment bhej sakta hai, offline condition me transaction local IndexedDB me save hoti hai, aur internet aate hi backend/PostgreSQL central ledger me sync ho jati hai.

1. Table Of Contents

> **Presentation opening script:** Good morning sir/ma'am. Our project is SyncPay, an offline-first QR based wallet. Normal payment apps fail when internet is weak, so our system allows a user to prepare and record payments offline using QR and local storage. When the device becomes online, pending transactions are automatically synced to the backend, where PostgreSQL works as the central ledger.

2. Project Overview

Problem

India me many users low network area, campus, shops, events, ya travel condition me payment karna chahte hain. Normal wallet/UPI flow internet ke bina fail ho sakta hai.

Solution

SyncPay wallet QR based flow provide karta hai. Online me immediate settlement hota hai. Offline me transaction signed payload ke form me local IndexedDB me save hota hai and baad me sync engine usko backend par bhejta hai.

Main Features

- Mobile number based signup/login with OTP.
- JWT based authenticated session.
- Wallet balance, available balance, offline limit and pending amount display.
- QR based receiver identity scan and payment QR scan.
- Online payment: direct backend ledger update.
- Offline payment: local IndexedDB queue and automatic sync when network returns.
- Admin panel: users, wallet credit, stats, transactions, support tickets.
- Support ticket and notifications modules.

Project Type

This is a full-stack web/PWA-style application. Frontend runs in browser using Next.js. Backend runs as Express.js REST API. PostgreSQL stores permanent ledger data. Redis/Twilio are optional services; development fallback is available.

3. Team Role-Wise Introduction

External presentation me team ko confidence ke saath role-wise bolna chahiye. Ye split natural lagega because har member apne module ko explain karega instead of sab kuch ek person bol raha ho.

Member	Suggested Role	Intro Lines For Presentation
Avinash	Backend, API, Database Integration	“I worked mainly on backend services, API routes, PostgreSQL connection, authentication, wallet transaction logic and offline sync validation. My part ensures that every payment finally reaches the central ledger safely.”
Aviral	Project Overview, Authentication Flow, Wallet Flow	“I focused on explaining the project problem statement, overall user journey, OTP authentication flow and wallet payment lifecycle. My part connects the business idea with the technical working.”
Shubhi	Frontend UI, User Flow, QR Screens	“I focused on the user-facing screens like onboarding, login, dashboard, send/receive payment flow and QR display. My work makes the app mobile-friendly and easy for users to understand.”
Abhinay	Offline Sync, Database Schema, Testing/Demo	“I handled the offline-first logic understanding, database schema explanation, sync flow testing and final demo preparation. My part connects the local pending queue with the backend ledger flow.”

> **Team intro script:** Our team divided the project into four main responsibilities: project/authentication flow, frontend experience, backend/ledger services, and offline sync/database validation. This helped us build SyncPay as a complete full-stack wallet application.

4. Technology Stack

Layer	Technology	Use In Project
Frontend	Next.js 14, React 18, TypeScript	Pages, routing, client UI, API proxy routes, QR screens, dashboard.
Styling	Tailwind CSS	Responsive wallet-style mobile UI and reusable utility classes.
State	Zustand	Auth, wallet and network status state.
QR	qrcode.react, jsQR	Generate QR code and scan QR from camera.
Offline Storage	IndexedDB via idb-keyval	Pending offline transactions queue on client device.
Backend	Node.js, Express.js	REST APIs for auth, wallet, transactions, sync, admin, support.
Database	PostgreSQL	Central ledger: users, wallets, transactions, devices, logs, notifications.
Auth/Security	JWT, bcryptjs, SHA-256 hash	JWT session, PIN hash, offline transaction signature/hash verification.
Optional Services	Redis, Twilio	OTP storage and SMS. In development Redis/SMS can fall back to logs/in-memory.

> **Viva answer:** We used Next.js because it gives routing, API proxy routes and React UI in one framework. Express is used for backend REST APIs. PostgreSQL is chosen because wallet ledger needs relational consistency and transactions.

5. Folder And File Structure

Root Folder

Path	Meaning
`frontend/`	Next.js client app, UI pages, stores, browser-side offline logic, API proxy routes.
`backend/`	Express API server, database connection, REST routes, auth middleware, schema scripts.
`syncpay-logo.png`	Project logo used in UI and documentation.
`RUN.md`, reports	Existing setup/report notes. Final viva notes are generated from actual code reading.

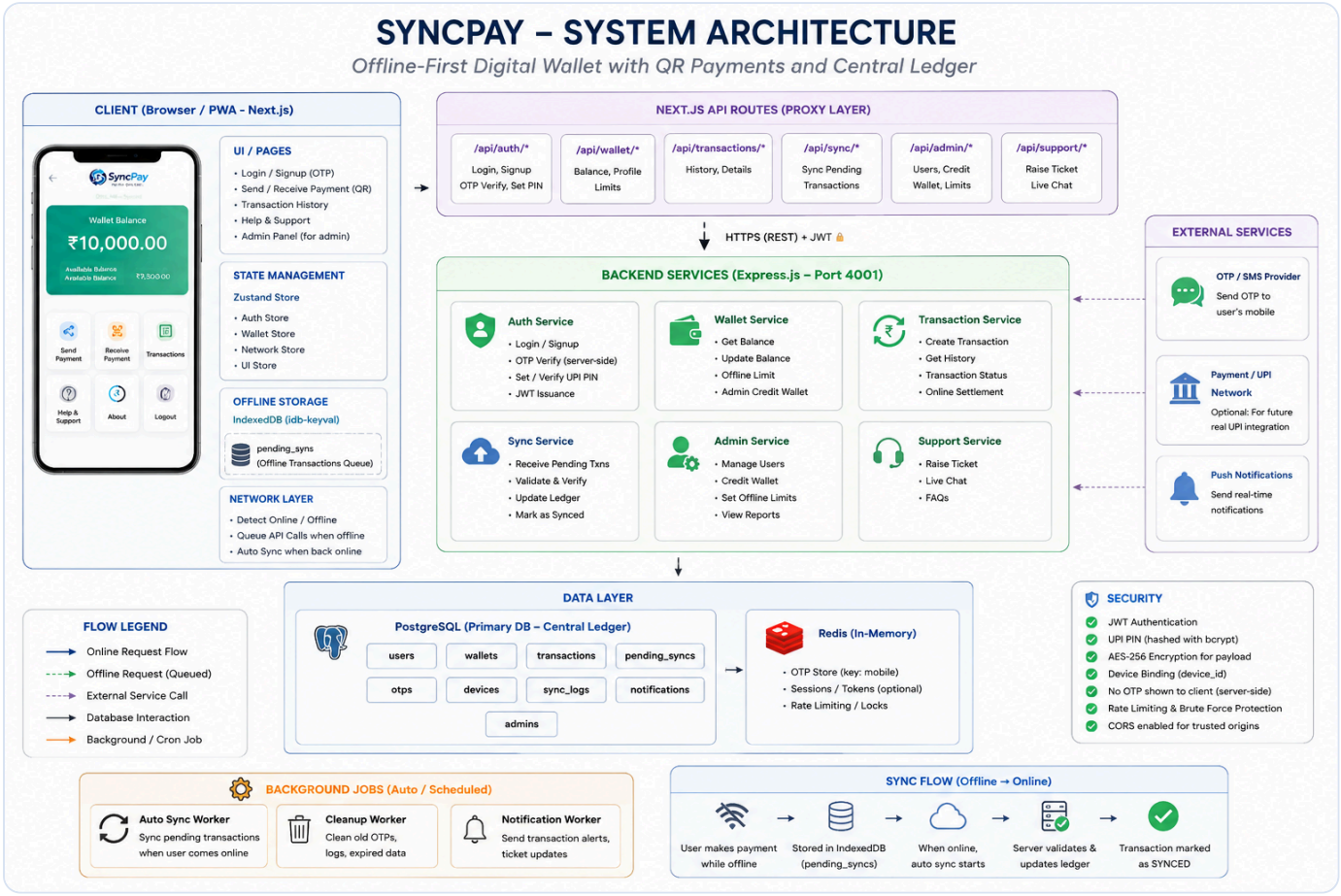
Frontend Important Files

File/Folder	Role
`src/app/layout.tsx`	Root layout. Global wrapper and CSS entry.
`src/app/providers.tsx`	Restores auth from localStorage and starts online sync listener.
`src/app/login`, `signup`, `otp`	Auth screens for mobile login/signup and OTP verification.
`src/app/dashboard/page.tsx`	Main wallet home: balance, offline spendable, pending amount, quick actions.
`src/app/send/*`	Sender flow: scan receiver QR, enter amount, confirm PIN, show payment QR.
`src/app/receive/page.tsx`	Receiver flow: shows identity QR and scans final payment QR.
`src/app/api/*`	Next.js server routes acting as proxy to Express backend.
`src/lib/idb.ts`	IndexedDB helper for saving, reading and marking pending offline transactions.
`src/lib/syncEngine.ts`	Auto-sync logic when network becomes online.
`src/lib/offlineSignature.ts`	Creates deterministic SHA-256 signature/hash for offline transaction payload.
`src/store/*.ts`	Zustand stores for auth, wallet and online/offline status.

6. Backend File Structure

File/Folder	Role
`src/server.js`	Express app entry. CORS, JSON middleware, route mounting, health route and error handler.
`src/config/index.js`	Reads environment variables like PORT, DATABASE_URL, JWT_SECRET, OTP expiry, offline limit.
`src/db/pool.js`	PostgreSQL connection pool using `pg`.
`src/db/schema.sql`	Main database schema: tables, types, indexes and wallet trigger function.
`src/db/init.js`	Runs schema/init database setup.
`src/db/seed.js`	Creates admin and demo users/wallets.
`src/db/redis.js`	OTP store. In development it uses in-memory Map instead of Redis.
`src/middleware/auth.js`	JWT verification and admin role guard.
`src/routes/auth.js`	Signup, login, OTP verify, user session, UPI PIN set.
`src/routes/wallet.js`	Get wallet, get receiver user, update offline limit.
`src/routes/transactions.js`	Transaction list, online send, receiver-side offline settlement.
`src/routes/sync.js`	Sender-side offline pending transaction sync endpoint.
`src/routes/admin.js`	Admin stats, users, wallet credit, transactions, pending syncs, reverse transaction.
`src/routes/support.js`	User support ticket creation.
`src/utils/crypto.js`	Payload hashing/signature verification and timestamp validity.

7. System Architecture



Architecture Explanation

1. **Client/browser:** Next.js pages run in browser. User interacts with login, dashboard, send, receive, history, help and admin pages.
2. **State management:** Zustand stores auth token/user, wallet data, and network status.
3. **Offline storage:** IndexedDB stores pending transaction payloads when internet is not available.
4. **Next.js API proxy:** Frontend calls `/api/...` routes. These proxy requests to Express backend using `NEXT_PUBLIC_API_URL`.
5. **Backend services:** Express routes validate JWT, verify PIN/signature, update wallets, create transactions and logs.
6. **Data layer:** PostgreSQL is the main source of truth. Redis/Twilio are optional for OTP/SMS.

> **Important for viva:** Do not say money is actually moved through UPI. This project simulates a wallet ledger. UPI/payment network integration is future scope.

8. Technical Deep Dive: How Everything Works

8.1 How OTP Works End-To-End

1. Mobile Enter
User enters 10 digit mobile number.

2. API Call
Frontend proxy calls backend `/auth/login` or `/auth/signup`.

3. OTP Generate
Backend creates random 6 digit OTP.

4. OTP Store
Redis stores OTP with TTL; dev uses in-memory Map.

5. Verify
User enters OTP, backend matches and returns JWT.

Important point: OTP frontend ko response me directly nahi bheja jata. Production me OTP SMS provider/Twilio ke through user ke phone par jayega. Development me `ALLOW_LOGGED_OTP=true` hone par backend terminal me OTP print ho sakta hai so testing easy ho jati hai.

```
Login request:
POST /auth/login
Body: { "mobile_number": "9504919122" }

Verify request:
POST /auth/verify-otp
Body: { "mobile_number": "9504919122", "otp": "123456" }

Successful verify response:
{ success: true, token: "JWT_TOKEN", user: { id, name, mobile_number, sync_id, role } }
```

OTP one-time use hota hai. Verification ke baad `deleteOTP(mobile)` call hota hai, so same OTP again use nahi kar sakte. OTP expiry `OTP_EXPIRE_SECONDS=300` hai, yani 5 minutes.

8.2 How Frontend Talks To Backend

Frontend browser directly Express backend ko call kar sakta hai, but CORS issue aa sakta hai because frontend `localhost:3000` par hai aur backend `localhost:4001` par. Is project me Next.js API routes proxy layer ki tarah kaam karte hain.

```
Browser UI
  -> /api/auth/login        (same-origin Next.js route)
  -> Next.js route.ts
  -> http://localhost:4001/auth/login   (Express backend)
  -> PostgreSQL / Redis
  -> Response back to browser
```

Iska benefit: frontend code clean rehta hai, backend URL `NEXT_PUBLIC_API_URL` se control hota hai, and browser-level CORS complexity reduce ho jati hai. Backend me CORS enabled hai for trusted/direct access, but normal app flow proxy se hota hai.

8.3 How QR Works

QR code basically ek JSON string ko image/pattern me encode karta hai. Project me `qrcode.react` QR generate karta hai and `jsQR` camera image se QR decode karta hai.

Receiver Identity QR

```
{
  "receiver_id": "receiver-user-uuid",
  "wallet_id": "receiver-user-uuid",
  "device_id": "web",
  "session_id": "SES-171...",
  "timestamp": 171...
}
```

Ye QR amount contain nahi karta. Iska purpose sirf receiver identity pass karna hai. Sender isko scan karta hai, backend se receiver validate hota hai, phir sender amount enter karta hai.

Sender Final Payment QR

```
{
  "txn_id": "transaction-uuid",
  "sender_id": "sender-user-uuid",
  "receiver_id": "receiver-user-uuid",
  "amount": 10000,
  "timestamp": 171...,
  "signature": "64-character-sha256-hex",
  "device_id": "web-abc"
}
```

Final QR offline mode me important hai. Sender ke device me transaction IndexedDB me save hoti hai, and receiver ko final QR scan karke proof/record milta hai. Jab network online hota hai, same payload backend par sync hota hai.

8.4 Online Payment Flow Diagram

```
Receiver Identity QR
      ↓
Sender scans receiver QR
      ↓
Sender enters amount + note
      ↓
Sender enters UPI PIN
      ↓
POST /transactions/send-transaction
      ↓
Backend verifies JWT + PIN + balance
      ↓
DB transaction: debit sender, credit receiver
      ↓
Status = SYNCED, payment success
```

8.5 Offline Payment Flow Diagram

```
Receiver shows Identity QR
      ↓
Sender scans QR while offline
      ↓
Sender enters amount + PIN
      ↓
Frontend creates txn_id + timestamp + SHA-256 signature
      ↓
Save in IndexedDB as PENDING_SYNC
      ↓
Sender shows Final Payment QR
      ↓
Receiver scans and stores pending incoming txn
      ↓
Internet returns
      ↓
syncEngine sends pending txns to backend
      ↓
Backend validates duplicate, timestamp, signature, balance
      ↓
PostgreSQL ledger updates, status = SYNCED
```

8.6 Why IndexedDB Is Used

`localStorage` sirf small string key-value data ke liye achha hota hai. Offline transaction queue structured object hai and durable browser database chahiye. Isliye IndexedDB use hua, and `idb-keyval` usko simple get/set API ke form me use karne deta hai.

9. Frontend Working

Frontend Flow

1. Login
User enters mobile, OTP request goes to backend.

2. Verify
OTP verify returns JWT and user info.

3. Dashboard
Wallet and pending amount are fetched/displayed.

4. Send/Receive
QR identity and payment QR are used.

5. Sync
Pending IndexedDB transactions sync online.

State And Storage

- `localStorage` : stores JWT as `syncpay_token` and device id.
- `sessionStorage` : temporary send/receive data like receiver id, amount, note, transaction id.
- `IndexedDB` : durable queue for offline pending transactions.
- `Zustand` : app-level memory state for auth, wallet and network.

Important Frontend Explanation

`providers.tsx` runs globally. It restores login from localStorage and calls `initSyncOnOnline()` . This means agar user offline payment karta hai, then jaise hi browser online event receive karta hai, sync engine pending transactions backend ko send karta hai.

`idb.ts` me functions like `savePendingTxns` , `getPendingTxns` , `markTxnSynced` , `markTxnFailed` hain. Ye offline transaction lifecycle maintain karte hain.

10. Backend Working

API Route Summary

Route	Purpose
`POST /auth/signup`	New user create, wallet create, OTP generate.
`POST /auth/login`	Registered user OTP login.
`POST /auth/verify-otp`	OTP verify, JWT issue.
`POST /auth/set-pin`	Set/change 6 digit UPI PIN after hashing with bcrypt.
`GET /wallet`	Current user's wallet balance.
`GET /wallet/user/:id`	Validate receiver after QR scan.
`POST /transactions/send-transaction`	Online payment and immediate wallet update.
`POST /transactions/settle-offline`	Receiver submits offline payment QR when online.
`POST /sync-transactions`	Sender device uploads pending offline queue.
`GET /admin/*`	Admin-only dashboard, users, transactions, support.

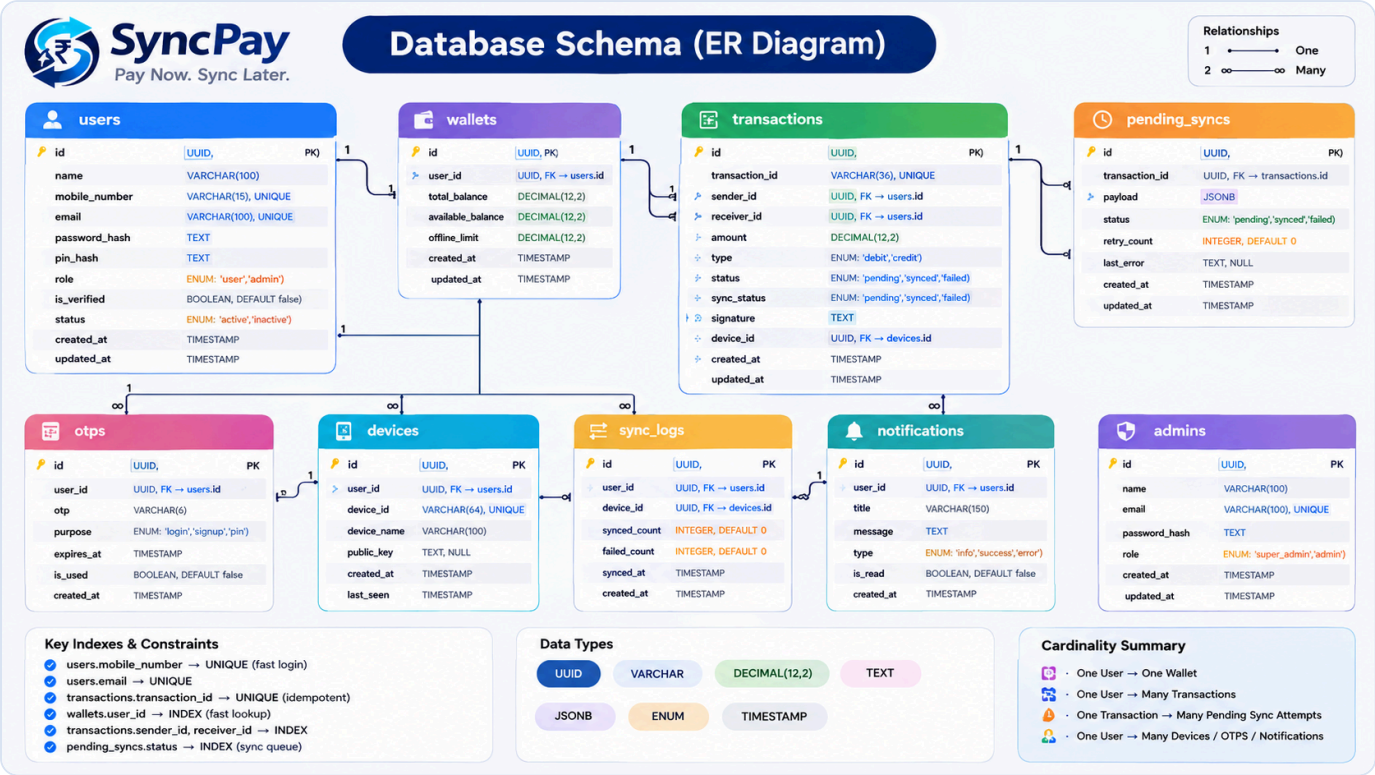
Auth Logic

JWT token carries `userId`, mobile, syncId and role. Backend middleware `requireAuth` validates token from Authorization header. Admin APIs first require auth and then `requireAdmin` checks role.

Wallet Ledger Logic

Amounts are stored in paise as integer/BIGINT, not floating rupees. Example: Rs 5000 = 500000 paise. This avoids decimal precision errors.

11. Database Schema



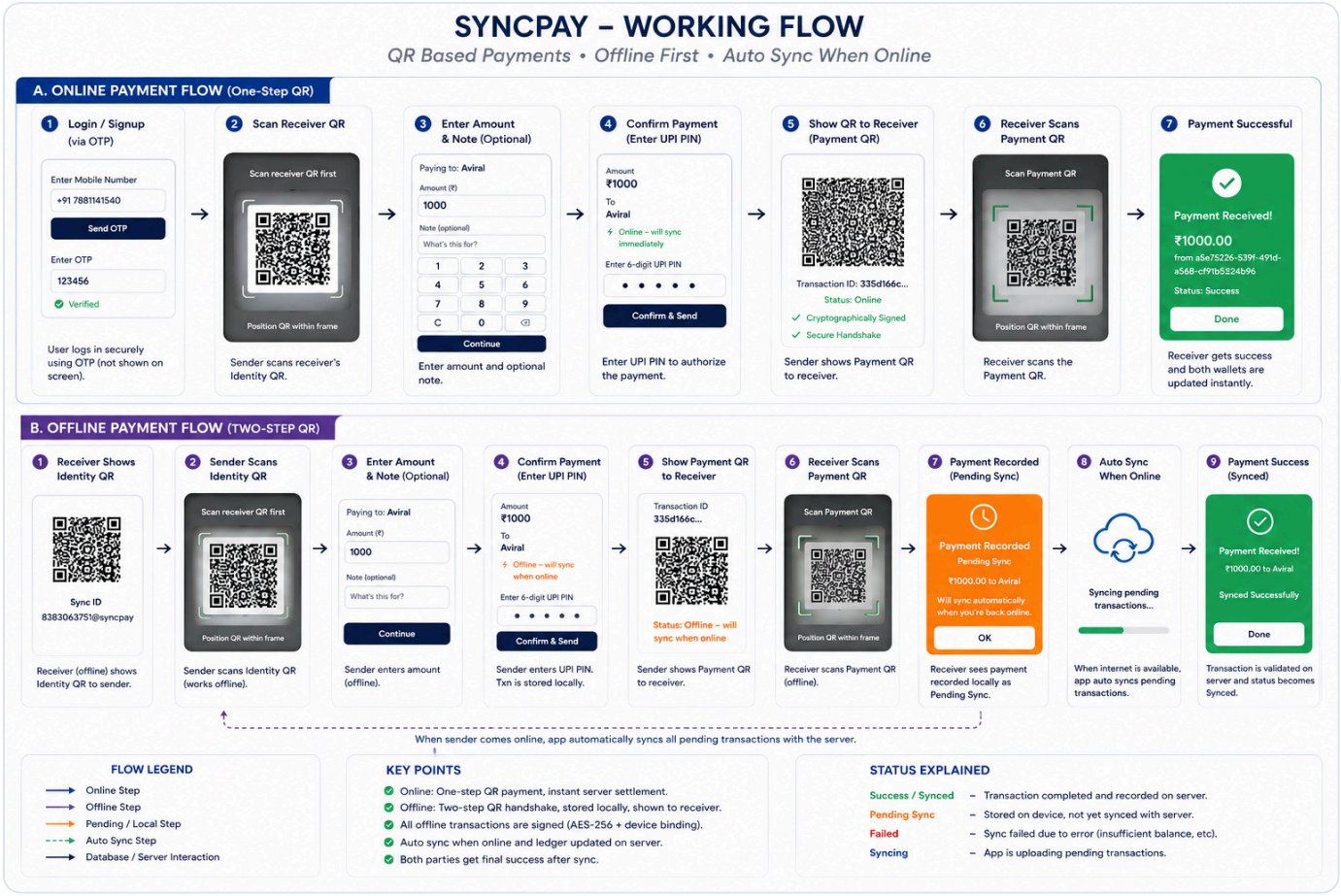
Main Tables In Actual Code

Table	Important Columns	Purpose
`users`	id, name, mobile_number, sync_id, pin_hash, role	Stores user profile and login identity.
`wallets`	user_id, total_balance, offline_reserved, available_balance, offline_limit	One wallet per user.
`transactions`	txn_id, sender_id, receiver_id, amount, status, sync_status, signature, device_id	Central ledger of all synced payments.
`devices`	device_id, user_id, device_name, last_active	Device binding/security tracking.
`sync_logs`	user_id, txn_count, status, details	Audit log for sync batches.
`notifications`	user_id, type, title, body, txn_id, read_at	User notification feed.
`support_tickets`	user_id, mobile_number, subject, message, status	Help/support tickets.
`fraud_alerts`	txn_id, user_id, reason, severity, resolved	Admin/fraud future module.

Relationships

- One user has one wallet: `wallets.user_id` is unique foreign key to `users.id`.
- One user can have many sent/received transactions.
- One user can have many devices, notifications, support tickets and sync logs.
- Transactions reference sender and receiver from users table.

12. Transaction Working Flow



Online Payment Flow

1. Receiver opens Receive page and shows identity QR.
2. Sender scans receiver QR, app validates receiver user.
3. Sender enters amount and optional note.
4. Sender enters UPI PIN.
5. Frontend calls `/api/transactions/send`, which proxies to backend `/transactions/send-transaction`.
6. Backend starts DB transaction, locks sender wallet row, checks balance, inserts transaction, debits sender and credits receiver.
7. Status becomes `SYNCED` and both wallets are updated.

Offline Payment Flow

1. Receiver shows identity QR even without internet.
2. Sender scans QR, enters amount and UPI PIN.
3. Frontend creates transaction id, timestamp, device id and SHA-256 signature.
4. Transaction is saved in IndexedDB with `PENDING_SYNC`.
5. Sender shows payment QR to receiver. Receiver scans it and stores incoming pending transaction.
6. When online, `syncEngine.ts` sends pending transactions to backend.

7. Backend verifies sender, duplicate transaction, timestamp, signature, receiver, balance, then updates ledger.

13. Installation And Run Steps

Prerequisites

- Node.js and npm installed.
- PostgreSQL installed locally, or hosted PostgreSQL configured in `DATABASE_URL`.
- Optional: Redis and Twilio. Development can work without them because OTP fallback exists.

Backend Setup

```
cd C:\Users\Avinash\Desktop\SyncPay\backend
npm install
npm run db:init
npm run db:seed
npm run dev
```

Backend current local port: `http://localhost:4001` according to current `backend/.env`.

Frontend Setup

```
cd C:\Users\Avinash\Desktop\SyncPay\frontend
npm install
npm run dev
```

Frontend: `http://localhost:3000`. Frontend environment points to backend using `NEXT_PUBLIC_API_URL=http://localhost:4001`.

Local Database Creation

```
psql -U postgres -c "CREATE DATABASE syncpay;"
```

Then update `backend/.env`:

```
PORT=4001
NODE_ENV=development
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/syncpay
REDIS_URL=redis://localhost:6379
JWT_SECRET=change-this-secret
JWT_EXPIRE=7d
OTP_EXPIRE_SECONDS=300
DEFAULT_OFFLINE_LIMIT=500000
ALLOW_LOGGED_OTP=true
ALLOW_OTP_WITHOUT_SMS=true
```

> **.env explanation:** `.env` stores configuration and secrets. We should not commit real passwords or JWT secrets. `.env.example` is only a template for teammates.

14. Database Commands And Sample Queries

Open PostgreSQL

```
psql -U postgres -d syncpay
```

If hosted DB is used, use pgAdmin or psql connection string from environment, but never show password publicly in viva.

Useful Viva Queries

```
-- 1. Show users
SELECT id, name, mobile_number, sync_id, role, created_at
FROM users
ORDER BY created_at DESC;

-- 2. Show wallet balances in rupees
SELECT u.name, u.mobile_number,
       w.total_balance / 100.0 AS total_rupees,
       w.available_balance / 100.0 AS available_rupees,
       w.offline_limit / 100.0 AS offline_limit_rupees
FROM wallets w
JOIN users u ON u.id = w.user_id;

-- 3. Show transaction history with sender/receiver names
SELECT t.txn_id, s.name AS sender, r.name AS receiver,
       t.amount / 100.0 AS amount_rupees,
       t.status, t.sync_status, t.created_at
FROM transactions t
JOIN users s ON s.id = t.sender_id
JOIN users r ON r.id = t.receiver_id
ORDER BY t.created_at DESC;

-- 4. Pending sync audit logs
SELECT user_id, txn_count, status, details, synced_at
FROM sync_logs
ORDER BY synced_at DESC;
```

Why Transactions Are Important

Online payment backend uses DB transaction with `BEGIN`, wallet row lock `FOR UPDATE`, and `COMMIT`. This prevents race conditions like two payments spending the same balance simultaneously.

15. Security And Reliability Points

- **JWT authentication:** Every protected API requires Bearer token.
- **UPI PIN:** PIN is never stored directly. It is hashed with bcrypt and only hash is stored.
- **OTP expiry:** OTP expiry is configured with `OTP_EXPIRE_SECONDS`. Development can log OTP for testing.
- **Offline signature:** Client builds deterministic payload and hashes with SHA-256. Backend recomputes hash and verifies it.
- **Timestamp validation:** Offline transactions have max age window. Current config default allows up to 7 days for offline sync.
- **Idempotency:** Duplicate transaction id is checked. If already synced with same sender/receiver, backend treats it safely as already settled.
- **Balance checks:** Sender wallet is checked before debit. Offline limit prevents unlimited offline spending.
- **CORS:** Backend allows frontend origin through CORS config.

> **Honest limitation:** Current project uses SHA-256 hash as demo signature. In production, proper asymmetric cryptography like Ed25519 public/private key pair should be used, and device public key should be stored server-side.

16. Admin And Support Module

Admin Role

Admin user is created in `seed.js`. After OTP login, JWT includes role `admin`. Dashboard redirects admin to admin panel.

Admin Features

- View total users, total transactions and pending sync count.
- List users with wallet balances.
- Credit wallet amount and set offline limit.
- View transactions and pending sync records.
- View support tickets.
- Reverse a synced transaction by moving money back and marking transaction failed.

Support

User can raise support ticket from help/support UI. Backend stores ticket in `support_tickets`. Admin can view tickets.

17. Viva Questions And Answers

Q1. What is SyncPay?

SyncPay is an offline-first QR based wallet system. It allows users to send and receive wallet payments. Online payments settle instantly, and offline payments are stored locally and synced later to the central PostgreSQL ledger.

Q2. Why did you use PostgreSQL?

Wallet and transaction data are relational. We need foreign keys, constraints, transaction safety, and consistent ledger updates. PostgreSQL gives ACID transactions, row locking and strong data integrity.

Q3. What happens when internet is off?

Frontend detects offline through browser network APIs. Instead of calling backend, it creates a transaction payload, signs/hashes it, saves it in IndexedDB as `PENDING_SYNC`, and displays QR to receiver. When internet returns, sync engine sends it to backend.

Q4. How do you prevent duplicate sync?

Each transaction has unique `txn_id`. Backend checks whether this id already exists. If it is already synced with same sender and receiver, it returns success as already settled. If duplicate mismatch happens, it fails.

Q5. How is balance updated?

Backend starts a DB transaction, locks sender wallet row, checks available balance, inserts transaction, debits sender wallet and credits receiver wallet, then commits. If any step fails, rollback happens.

Q6. What is offline limit?

Offline limit is maximum amount a user can spend while offline. Frontend checks pending offline amount plus new amount against this limit. Admin can credit wallet and set offline limit.

Q7. What is JWT?

JWT is a signed token used for session authentication. After OTP verification backend issues JWT. Frontend sends it in Authorization header for protected APIs.

Q8. What is IndexedDB?

IndexedDB is browser-side persistent storage. We use it through idb-keyval to save pending transactions because localStorage is not suitable for structured offline queue data.

Q9. What are project limitations?

Real banking/UPI integration is not implemented. Offline security uses demo hash signature; production should use true public/private key signing. SMS service is optional and development can log OTP.

18. Member-Specific Viva Questions

Avinash - Backend/API/Database Viva

Question	Answer
What was your main contribution?	I handled backend APIs, PostgreSQL integration, JWT authentication, wallet transaction update logic and offline sync validation.
How does backend protect APIs?	Protected routes use `requireAuth`. It reads Bearer token, verifies JWT secret, and attaches user data to `req.user`. Admin routes also check `role === 'admin'`.
How does online payment update wallet safely?	Backend starts a DB transaction, locks sender wallet row using `FOR UPDATE`, checks balance, inserts transaction, debits sender, credits receiver and commits. If any error occurs, rollback happens.
Why store amount in paise?	Integer paise avoids floating-point precision issue. For example Rs 100 is stored as 10000.
How does offline sync validation happen?	Backend checks sender ownership, duplicate transaction id, timestamp validity, signature/hash, receiver existence and sender balance before ledger update.

Aviral - Project Overview/Auth/Wallet Viva

Question	Answer
What was your main contribution?	I prepared the overall project explanation, problem statement, OTP authentication flow, wallet lifecycle and user journey from login to payment success.
What problem does SyncPay solve?	It solves the issue of payment failure in weak or no internet areas by allowing offline transaction recording and later server sync.
How does OTP login work?	User enters mobile number, backend generates 6 digit OTP, stores it temporarily using Redis or in-memory store, and after correct OTP verification backend returns JWT token.
What is the role of wallet in this project?	Wallet stores total balance, available balance, offline limit and pending/settled transaction impact. It is linked one-to-one with each user.
How will you explain the project in one minute?	SyncPay is an offline-first QR wallet where online payments settle instantly and offline payments are stored locally, then validated and synced to PostgreSQL central ledger when network returns.

Shubhi - Frontend/UI/QR Viva

Question	Answer
What was your main contribution?	I worked on frontend screens, mobile-friendly user flow, dashboard, send/receive pages and QR generation/scanning experience.
How is QR generated?	We use `qrcode.react`. A JSON payload is converted to string using `JSON.stringify` and passed to `QRCodeSVG`.
How is QR scanned?	The app opens camera with browser media APIs, draws video frame on canvas and `jsQR` decodes QR data from image pixels.
What does dashboard show?	Dashboard shows total wallet balance, offline spendable amount, offline limit remaining, pending outgoing/incoming sync amount and quick actions.
How is login state maintained?	JWT token is stored in localStorage and app state is managed with Zustand. Provider restores user session by calling `/api/auth/me`.

Abhinay - Offline Sync/Schema/Demo Viva

Question	Answer
What was your main contribution?	I focused on offline sync understanding, database schema explanation, transaction flow testing and demo preparation.
Where are offline transactions stored?	They are stored in browser IndexedDB database `syncpay-offline`, store `pending_transactions`, with transaction id as key.
When does sync run?	`initSyncOnOnline` listens to browser online event. When internet returns, `syncPendingTransactions` sends pending transactions to backend.
What are main DB tables?	Main tables are users, wallets, transactions, devices, sync_logs, notifications, support_tickets and fraud_alerts.
How will you demo database?	I can run SQL queries to show users, wallet balances and transaction history joined with sender/receiver names.

19. Team Presentation Split

Teammate	Topic To Explain	Key Lines
Avinash	Backend + Database Integration	Express APIs, JWT middleware, wallet update, PostgreSQL ledger, sync endpoint validation.
Aviral	Introduction + OTP/Wallet Flow	Problem statement, solution overview, OTP authentication, wallet lifecycle and online payment explanation.
Shubhi	Frontend + QR UI	Next.js pages, dashboard, send/receive screens, QR generation/scanning, user-friendly mobile flow.
Abhinay	Offline Sync + Demo + Queries	IndexedDB pending queue, auto sync, schema relationships, sample SQL queries and final demo/security/future scope.

Final Closing Script

To conclude, SyncPay demonstrates a practical offline-first wallet design. The main idea is that the client can safely queue transactions during network failure, and the backend later validates and settles them in a central ledger. This makes the project useful for low-connectivity situations and also shows full-stack concepts like authentication, database transactions, QR handling and offline sync.